RackWire

DEVELOPER MANUAL · 21 OCTOBER 2026

Contracts, boundaries, and verification

Implementation reference for the current standalone planner, optional discovery service, and SolariNet embedding boundary.

Observed base revision: RackWire `5f29f59`; this manual describes its local integration candidate, which is not a production release.

01 · ARCHITECTURE

Three boundaries



RackWire plans network, power, and interconnect systems across supported devices; rack layouts are optional. The planner runs directly from `file://` for core work. Its optional standalone discovery service is independent of SolariNet: it neither reads SolariNet's SoR nor shares a database. In the SolariNet dashboard embed, the Network discovery action routes to SolariNet's authenticated, bounded scan queue and candidate review screen. Physical scanner-ingest data enters through the dashboard preview/apply importer into shared SoR inventory; RackWire resolves its barcodes when the SoR model has a RackWire device profile.

Part	Authoritative files	Runtime
Planner	`index.html`, `device-library.js`, `devices/*.js`, `live-adapter.js`, `support.js`	Browser React/ReactDOM/Babel copies under `vendor/`; no build step.
Discovery	`backend/rackwire_discovery/wsgi.py`, `api/routes.py`, `scan/`, `infer/`, `store.py`	Python WSGI, SQLite; gunicorn for hosted service.
SolariNet	`dashboard/api/routes/rackwire.php`, `screens-rackwire.jsx`, SoR migration 020	PHP dashboard; pinned RackWire git submodule.

02 · BROWSER RUNTIME

No bundler; pinned runtime

`index.html` loads `support.js`, the library engine, vendor device modules, and the telemetry adapter as browser scripts. The generated `support.js` resolves its React, ReactDOM, and Babel resources to vendored local files via `window.__resources`. The runtime source tree is absent from this repository; treat `support.js` as pinned output, not an editable source file.

The documented host API is `window.RackWire` and the embed message bridge. Preserve their method names, message schema, and `rackwire.project.v2` storage schema. Query flags include `?embed=1`, `?view=table`, and `?live=simulator`.

> **Runtime change gate.** A change to `support.js`, the resource mappings, or `vendor/` requires the pinned checksum check and an offline cold load. The current generated runtime evaluates app code dynamically, so hosted CSP needs an explicit compatibility review.

03 · DATA CONTRACT

Plan snapshots and interchange

The planner stores the working plan under `rackwire.project.v2` in browser storage, with undo/redo, named snapshots, JSON plan import/export, CSV exchange, and a printable punch list. A placed device snapshots its library definition; later catalog edits do not silently change an existing plan. Keep plan migration logic compatible when adding persisted device or group fields.

Observed: `window.RackWire` exposes a 23-method host API. The embedded parent uses the postMessage bridge. Standalone operation does not require either host call or SolariNet session. The SoR plan service can store shared plans and versions when present; offline use remains browser-local.

04 · DEVICE LIBRARY

Curated seed and discovered models

`device-library.js` owns layout and indexing. Each `devices/<vendor>.js` is a plain script calling `window.RW.addDevices([...])`; modules do not perform network I/O. The reviewed `devices/SPEC.md` records 244 curated devices across 20 vendors, with required stable ID, vendor, name, model, kind, flavor, rack units, draw, and groups. Device values must come from vendor specifications, and the browser validator is `devices/validate.html`.

The discovery backend has a separate mutable SQLite library. Its confirmed models merge into the browser catalog and are not written into the static vendor modules. SolariNet can separately merge SoR models through `/api/rackwire/library` when embedded.

05 · TELEMETRY

Frames are optional overlays

`RWLive.register(name, factory)` accepts a source returning `start()` and `stop()`. The context provides `getPlan()`, `emit(frame)`, `setStatus()`, and source config. Built-ins are `simulator`, `solarinet-rest`, and `solarinet-ws`; the SolariNet contract uses REST polling. A missing field remains absent rather than becoming a zero measurement.

```
{ ts: 1730000000000,
  devices: { "device-key": { reachable: true, loadW: 143.2 } },
  ports: { "device-key|port-id": { link: true, negotiatedGbps: 2.5 } } }
```

Device key resolution is `liveKey`, then `assetId`, then name, case-insensitive. For SolariNet, interface names map through a per-device `ifMap` to internal port IDs. The PHP endpoint omits fields without a backing source, including PoE watts and optical measurements.

06 · DISCOVERY BACKEND

Separate scan and confirmation pipeline

The planner's Discover panel calls `/api/discovery` on the standalone RackWire origin. `POST /scan` starts a background run; API reads expose progress and candidates. ARP and ping produce liveness evidence; nmap, SNMP, LLDP, OUI, and optional Claude inference enrich it. The operator confirms or rejects candidates before associating an existing catalog entry or saving a validated new model.

Surface	Contract
Health and scan	`GET /api/discovery/health` reports status and a CSRF token. `POST /scan`, `GET /scan/{id}`, and `POST /scan/{id}/cancel` manage runs.
Review	`GET /devices`, `GET /devices/{id}/candidate`, `POST /devices/{id}/infer|confirm|reject|associate`.
Library and topology	`GET|POST /library`, `POST /plan/build`, `GET /topology`.

`store.py` is the SQLite access layer. Scanner modules return facts without database writes. `api/routes.py` validates HTTP payloads; `wsgi.py` enforces a CSRF token on every mutation. The subnet allowlist denies scans when empty. Keep system-tool calls bounded and use argument lists, never shell strings.

07 · SOLARINET INTEGRATION

Versioned dependency boundary

SolariNet pins this repository as `dashboard/public/rackwire`. A same-origin iframe uses the existing host bridge; its PHP route exposes plan/version CRUD, hardware-model library, telemetry, and a dry-run or transactional commit to the SoR. Migration 020 holds RackWire plan, model-group, and power-circuit structures. The dashboard inventory editor writes model connection groups through the guarded `POST /api/rackwire/library/{modelId}/groups` route; profile rows owned by another source are protected from overwrite. The embedded dashboard exposes `Scan inventory barcode` and sends the origin-checked `openScan` host command to the frame. Barcode lookup uses the shared inventory API with the dashboard session cookie and `X-Solari-CSRF`; RackWire refreshes one stale token and preserves the scanned asset tag and serial on the placed device. Planned interconnects carry RackWire provenance; LLDP observations are read-only context.

SolariNet's RackWire API is distinct from standalone discovery. The embedded discovery action navigates to SolariNet's authenticated bounded scan queue; it does not call the standalone discovery API. Discovery adoption creates or updates a monitoring host and does not create a physical hardware unit. Do not assume a standalone RackWire origin has a SolariNet session. Browser-local planning must still work if both optional services are unavailable.

08 · BOUNDARIES AND RELEASE POSTURE

Keep intranet limits visible

Discovery is documented for a single-operator intranet deployment. Its mutation CSRF token is signed and time-bounded, but full multi-user authentication, per-operator scope, scan rate limiting, and tighter session binding remain public-release blockers. The WSGI app listens on loopback behind a TLS proxy in the documented deployment. Never put credentials into a plan, log, documentation artifact, or repository.

The generated browser runtime has message and dynamic-evaluation constraints; review framing, origin handling, and CSP before exposing a hosted public app. The SolariNet integration candidate has its own authorization and CSRF policy, which must be tested at the PHP boundary.

09 · VERIFICATION

Run the boundary checks

Check	Command or target	Meaning
Discovery backend	`cd backend && python3 -m pytest -q`	Scanner, inference, store, API, and CSRF behavior.
Runtime provenance	`sha256sum -c docs/runtime-checksums.sha256`	Vendored browser artifact pins.
Catalog	Open `devices/validate.html` in a browser.	Check static vendor modules against the device spec.
SolariNet boundary	Run `tests/dashboard/test_rackwire_api.php` and `test_rackwire_ui.js` in the parent repo.	Route and token bridge checks; not a live browser smoke.
Standalone browser	Run `node tests/browser/test_rackwire_standalone.cjs` from the SolariNet root with Playwright 1.64.1 and Chromium available.	Exercises file-origin boot, accessible dialog focus, local plan persistence, navigation, and a 390 px toolbar; it does not prove authenticated iframe or SoR commit behavior.

Before a release, exercise `file://` offline boot, a hosted planner with discovery, the same-origin iframe, plan save/load, telemetry, barcode lookup, and dry-run commit against disposable data. None of these checks alone proves the other boundary.

Observed, inferred, unverified

Observed: Source in this local integration candidate defines the planner, discovery routes, CSRF gate, telemetry sources, and SolariNet API seams described above. Offline standalone browser acceptance passed, and separate SolariNet browser acceptance covers scanner import → SoR → RackWire, barcode lookup, plan persistence, and the bounded Discovery handoff against disposable databases.

Inferred: A deployed host with matching configuration should make discovery and SolariNet features available; source inspection and scratch acceptance do not establish deployed availability.

UNVERIFIED: Live RackWire hostname and discovery service, browser camera permissions, deployed SolariNet iframe/API behavior, live production SoR commit, public hosting hardening, and release ownership. This manual is an integration-candidate reference, not a declaration that those flows are deployed.